

Tutoriel complet de QGIS : interface, symbologie, boîte à outils de traitement, modèleur graphique, extensions et automatisation par lot, en trois parcours de niveau.

SOMMAIRE

1. PYQGIS AVANCÉ : SCRIPTS AUTONOMES HORS INTERFACE

2. DÉVELOPPER UN ALGORITHME PROCESSING OU UNE EXTENSION MINIMALE

3. CONNEXION À POSTGIS DEPUIS QGIS

4. AUTOMATISER EN LIGNE DE COMMANDE : QGIS_PROCESS

5. INTEROPÉRABILITÉ : GDAL/OGR ET L'ÉCOSYSTÈME HORS QGIS

6. REPRODUCTIBILITÉ : VERSIONNER UN PROJET QGIS

Le module Le Compas présente les SIG en général et QGIS en particulier, mais reste volontairement une vue d'ensemble. Cette salle est un vrai tutoriel : installer QGIS, apprendre son interface bouton par bouton, produire une carte imprimable, puis, plus loin, automatiser des traitements entiers par script. Trois pistes complètes ci-dessous (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin, sur QGIS 3.x en version LTR (« Long Term Release », la version stable recommandée, par exemple 3.34 « Prizren »).

VOIR AUSSI : AVANT DE COMMENCER : À QUOI SERT UN SIG

Le module Le Compas pose le vocabulaire (couche, symbologie, opérations spatiales) que ce tutoriel QGIS suppose déjà connu, ou qu'il redéfinit au fil de l'eau.

1. PYQGIS AVANCÉ : SCRIPTS AUTONOMES HORS INTERFACE

Au-delà de la console interne (piste Licence/BUT), PyQGIS s'utilise aussi en dehors de toute interface graphique : un script Python autonome, lancé depuis un terminal ou planifié (cron, tâche planifiée), qui initialise QGIS en mode « headless » (sans fenêtre) pour traiter un dossier entier de fichiers sans jamais ouvrir l'application.

INITIALISER QGIS EN MODE HEADLESS

```
from qgis.core import QgsApplication
QgsApplication.setPrefixPath('/chemin/vers/qgis', True) qgs =
QgsApplication([], False) qgs.initQgis() # ... traitements ...
qgs.exitQgis()
```

Le second argument False de QgsApplication désactive l'interface graphique. setPrefixPath() pointe vers l'installation QGIS locale (variable selon le système). initQgis()/exitQgis() encadrent tout le script : oublier exitQgis() peut laisser des ressources non libérées en fin d'exécution.

TRAITER UN DOSSIER DE FICHIERS EN LOT, PAR SCRIPT

```
import glob
for path in glob.glob('/donnees/*.gpkg'): layer =
QgsVectorLayer(path, 'temp', 'ogr')
processing.run('native:buffer', {'INPUT': layer, 'DISTANCE':
200, 'OUTPUT': path.replace('.gpkg', '_buffer.gpkg')})
```

glob.glob() liste tous les fichiers correspondant au motif ; la boucle applique le même traitement à chacun, sans limite de nombre et sans risque d'oubli — l'automatisation complète de ce qu'ébauchait le traitement par lot de l'interface (piste Licence/BUT, section 1).

2. DÉVELOPPER UN ALGORITHME PROCESSING OU UNE EXTENSION MINIMALE

Au-delà de scripter des algorithmes existants, PyQGIS permet d'en écrire de nouveaux, intégrés à la boîte à outils au même titre que ceux fournis par défaut, ou de développer une extension complète avec sa propre interface.

- Un algorithme Processing personnalisé hérite de `QgsProcessingAlgorithm`, déclare ses paramètres d'entrée/sortie dans `initAlgorithm()`, et implémente sa logique dans `processAlgorithm()` — une fois enregistré via un `QgsProcessingProvider`, il apparaît dans la boîte à outils comme n'importe quel algorithme natif, utilisable aussi en script ou dans le Modeleur graphique
- L'extension « Plugin Builder », installable depuis le gestionnaire d'extensions, génère le squelette complet d'une nouvelle extension QGIS (structure de fichiers, métadonnées, interface minimale) à partir de quelques questions, plutôt que de partir d'une page blanche
- L'extension générée se recharge en développement avec « Plugin Reloader » (autre extension), qui évite de redémarrer QGIS entier à chaque modification du code

REMARQUE

SCRIPT, ALGORITHME PROCESSING OU EXTENSION : TROIS NIVEAUX D'INTÉGRATION

Un script autonome (section 1) convient à un traitement ponctuel ou planifié, sans besoin d'interface. Un algorithme Processing personnalisé convient dès qu'un traitement doit être réutilisé par d'autres, combiné dans un Modeleur graphique, ou appelé en ligne de commande (section 4). Une extension complète ne se justifie que lorsqu'une vraie interface graphique dédiée (panneaux, dialogues, outils de carte personnalisés) apporte quelque chose qu'un simple algorithme ne peut pas offrir.

- Tester un algorithme personnalisé sur des cas limites avant de le diffuser : géométrie vide, valeur de champ nulle, couche sans entité sélectionnée — les mêmes catégories d'erreurs qu'un script Python classique, mais plus faciles à ignorer derrière une interface qui semble fonctionner sur le premier exemple testé
- Le cadre de test officiel de QGIS (QGIS Testing Framework, basé sur `unittest`/`pytest`) permet d'écrire de vrais tests automatisés pour une extension, exécutés sans interface graphique, sur le même principe qu'un test unitaire pour n'importe quel logiciel

3. CONNEXION À POSTGIS DEPUIS QGIS

Au-delà d'un fichier local (GeoPackage, Shapefile), QGIS se connecte directement à une base de données spatiale PostGIS, ce qui change l'échelle possible d'un projet : plusieurs utilisateurs simultanés, des volumes de plusieurs millions d'entités, et des requêtes exécutées côté serveur plutôt que rapatriées intégralement en mémoire.

1. Gestionnaire de sources de données (Ctrl+L, ou Couche > Ajouter une couche > Couche PostGIS...) : créer une nouvelle connexion (hôte, port, base, identifiants), puis parcourir les tables/vues disponibles et les charger comme des couches classiques
2. Une fois connectée, une couche PostGIS se comporte comme n'importe quelle couche vecteur dans QGIS : symbologie, table attributaire, Processing s'appliquent sans différence visible
3. DB Manager (Base de données > DB Manager...) : exécuter directement une requête SQL spatiale sur la base connectée, et charger son résultat comme couche temporaire — utile pour une jointure ou un filtre complexe que l'interface QGIS n'exprime pas nativement

EXEMPLE DE REQUÊTE DANS DB MANAGER

```
SELECT nom, geom FROM communes WHERE ST_Intersects(geom,  
ST_Buffer((SELECT geom FROM rivières WHERE nom = 'Durance'),  
200));
```

Exécutée côté serveur PostGIS, cette requête ne rapatrie vers QGIS que le résultat déjà filtré, pas la table entière — un avantage décisif face à un fichier local dès que le volume dépasse quelques dizaines de milliers d'entités.

VOIR AUSSI : ALLER PLUS LOIN : POSTGIS, INDEX SPATIAUX ET PERFORMANCE

Le module La Base détaille PostGIS en profondeur : index GiST, EXPLAIN ANALYZE, topologie, exposition web d'une base spatiale.

4. AUTOMATISER EN LIGNE DE COMMANDE : QGIS_PROCESS

qgis_process, installé avec QGIS, exécute n'importe quel algorithme de Processing directement depuis un terminal, sans jamais ouvrir l'interface graphique — la brique qui permet d'intégrer un traitement QGIS dans un pipeline plus large (un script shell, une tâche planifiée, une chaîne de traitement continue).

LISTER PUIS EXÉCUTER UN ALGORITHME EN CLI

```
qgis_process list qgis_process run native:buffer --  
INPUT=communes.gpkg --DISTANCE=200 --OUTPUT=buffer.gpkg
```

qgis_process list affiche l'identifiant exact de chaque algorithme disponible (le même identifiant technique qu'utilise processing.run() en PyQGIS) ; run l'exécute avec ses paramètres passés en ligne de commande, sans dépendance à une session QGIS graphique ouverte.

REMARQUE

INTÉGRER QGIS_PROCESS DANS UN PIPELINE

Un serveur qui reçoit chaque nuit une nouvelle image satellite peut déclencher automatiquement une chaîne qgis_process (calcul d'indice, statistiques de zone, export) via une tâche planifiée, sans qu'un opérateur n'ouvre jamais QGIS lui-même : le traitement par lot de l'interface (piste Licence/BUT) et le script Python (section 1) convergent ici vers un même objectif, l'automatisation complète, par une troisième voie qui ne nécessite même pas de connaître PyQGIS.

CHAÎNER QGIS_PROCESS DANS UN SCRIPT SHELL

```
#!/bin/bash for f in /donnees/entrantes/*.tif; do qgis_process  
run gdal:rastercalculator --INPUT_A="$f" --BAND_A=1 --  
FORMULA="A*0.0001" --OUTPUT="/donnees/sorties/$(basename  
"$f")" done
```

Un script shell classique orchestre plusieurs appels qgis_process, un par fichier : la même logique que le script Python de la section 1, mais sans écrire une seule ligne de PyQGIS, en s'appuyant uniquement sur le CLI Processing.

5. INTEROPÉRABILITÉ : GDAL/OGR ET L'ÉCOSYSTÈME HORS QGIS

La plupart des formats que QGIS lit ou écrit (Shapefile, GeoTIFF, GeoPackage...) passent en réalité par GDAL (raster) et OGR (vecteur), deux bibliothèques logicielles indépendantes de QGIS, invisibles depuis l'interface mais qui font le même travail derrière d'autres outils : ogr2ogr (conversion et reprojection en ligne de commande), gdal_translate, mais aussi les bibliothèques Python (GeoPandas, rasterio) ou R (sf) hors de toute interface SIG.

REMARQUE

LE MÊME FICHIER, PLUSIEURS OUTILS

Un GeoPackage produit par QGIS s'ouvre sans conversion dans R (bibliothèque sf) ou dans un script Python autonome (GeoPandas) : les trois environnements s'appuient sur le même GDAL/OGR en coulisses, ce qui garantit une compatibilité qui n'a rien d'un hasard. Choisir QGIS, R ou un script Python hors QGIS pour une étape donnée d'un pipeline est souvent affaire de commodité, pas de format imposé.

- Un script GeoPandas peut préparer une donnée que QGIS affichera ensuite (mise en page finale, section Lycée) — l'inverse fonctionne tout aussi bien
- R, via le paquet sf (basé sur GDAL/OGR/PROJ, les mêmes bibliothèques que QGIS), prend le relais pour une analyse statistique spatiale poussée (régression spatiale, modèles) que QGIS n'offre pas nativement
- Un environnement de développement dédié (VS Code) reste l'outil de référence pour écrire, déboguer et versionner ces scripts, qu'ils appellent PyQGIS, GeoPandas ou R

PROGRAMMATION-R

- Langage dédié à la statistique, avec sf pour la donnée spatiale
- Pertinent dès qu'une analyse statistique dépasse ce qu'une interface graphique propose

VSCODE

- Éditeur de code générique, pas spécifique à la géomatique
- Environnement de travail réel pour tout script PyQGIS, GeoPandas ou R au-delà de quelques lignes

VOIR AUSSI : LA STATISTIQUE SPATIALE EN R

La salle De l'Instrument et du Nombre reprend l'analyse spatiale depuis le langage R, avec sf et tmap.

VOIR AUSSI : L'ÉDITEUR POUR ÉCRIRE ET DÉBOGUER CES SCRIPTS

La salle De la Plume et du Texte détaille VS Code : extensions Python/Jupyter, débogueur, Git intégré.

APPROFONDISSEMENT

6. REPRODUCTIBILITÉ : VERSIONNER UN PROJET QGIS

Un projet de recherche ou une chaîne de production répétée dans le temps gagne à être reproductible : n'importe qui (soi-même six mois plus tard, ou un collègue) doit pouvoir reconstituer exactement le même résultat à partir des mêmes données brutes.

.QGZ (PAR DÉFAUT)

- Archive compressée (format ZIP), plus légère sur disque
- Contenu binaire : un outil de version (Git) affiche « fichier modifié » sans jamais pouvoir montrer ce qui a changé à l'intérieur

.QGS (XML)

- Simple fichier texte structuré, plus volumineux
- Un différentiel Git (git diff) reste lisible ligne par ligne : quel paramètre de symbologie a changé, quelle couche a été ajoutée

- Pour un projet suivi dans Git, préférer explicitement le format .qgs (Projet > Enregistrer sous..., choisir le type de fichier) malgré sa taille plus importante, justement pour profiter d'un historique de modifications lisible
- Ne jamais versionner les données volumineuses elles-mêmes dans le même dépôt Git qu'un projet QGIS : un dépôt Git n'est pas conçu pour de gros fichiers binaires répé-

tés (raster, Shapefile volumineux) ; les référencer par chemin relatif et les stocker à part (serveur de données, Git LFS) reste la pratique correcte

- Documenter un traitement : l'historique de Processing (piste Licence/BUT, section 1) s'exporte en script Python reproductible ; un modèle du Modeleur graphique s'enregistre comme fichier .model3, lui aussi versionnable en Git au même titre qu'un script

EXEMPLE

UN PIPELINE REPRODUCTIBLE DE BOUT EN BOUT

Un dépôt Git qui contient un projet .qgs, un script qgis_process ou PyQGIS documentant chaque étape de traitement, et un fichier listant précisément les données brutes attendues (sans les données elles-mêmes) permet à quiconque clone ce dépôt de reconstruire le résultat final à l'identique — la même exigence de reproductibilité qu'un notebook R Markdown versionné (salle De l'Instrument et du Nombre) ou qu'un script Python versionné sous VS Code (salle De la Plume et du Texte).

- Bilan — à retenir : PyQGIS headless (QgsApplication, initQgis/exitQgis) traite un dossier entier sans interface ; un algorithme Processing personnalisé (QgsProcessingAlgorithm) ou une extension (Plugin Builder) prolongent un simple script réutilisable ; PostGIS se connecte via le Gestionnaire de sources ou DB Manager, avec SQL spatial exécuté côté serveur ; qgis_process exécute Processing en CLI, intégrable à un pipeline planifié ; GDAL/OGR sous-tend QGIS, GeoPandas et R (sf), d'où leur interopérabilité native ; .qgs (XML) se versionne lisiblement en Git, .qgz beaucoup moins ; les données volumineuses ne se versionnent jamais directement dans le même dépôt qu'un projet QGIS.

VOIR AUSSI : CONTINUER : BASES DE DONNÉES SPATIALES EN PROFONDEUR

Le module La Base reprend PostGIS depuis les index spatiaux jusqu'à l'exposition web d'une base spatiale complète.